

Progetto Architetture 2025

CALCOLO APPROSSIMATO DEI K VICINI

Fabrizio Angiulli, Fabio Fassetti

a.a. 2025/2026

Gruppo 6

Benedetta Calonico (*mat. 276653*)

Francesco Pio Ceraudo (*mat. 276643*)

Rosario Chiappetta (*mat. 277011*)

Indice

1	Introduzione e richieste del progetto	5
1.1	Richieste della traccia e vincoli di consegna	5
1.2	Panoramica dell'algoritmo (QuantPivot)	6
1.2.1	Il problema dei K-Nearest Neighbors	6
1.2.2	Quantizzazione dei vettori (top- x sparsa)	6
1.2.3	Distanza approssimata	7
1.2.4	Indicizzazione tramite pivot (fase fit)	7
1.2.5	Querying e pruning (fase predict)	7
1.3	Struttura del progetto	7
2	Compilazione, esecuzione e validazione sperimentale	10
2.1	Dipendenze e ambiente di esecuzione	10
2.2	Installazione editabile del package	11
2.3	Integrazione Python/C: wrapper e vincoli di allineamento	11
2.3.1	Check di allineamento nel wrapper	11
2.4	Formato .ds2 e caricamento in memoria (script di test)	12
2.5	Pulizia degli artefatti di build	12
2.6	Modalità di compilazione: baseline C e ASM	12
2.6.1	Abilitazione ASM a build-time	13
2.7	Esecuzione e validazione: test standard + compare_results	13
2.7.1	Versione 32-bit: baseline C	13
2.7.2	Versione 32-bit: ASM SSE	13
2.7.3	Versione 64-bit: baseline C	14
2.7.4	Versione 64-bit: ASM AVX	14
2.7.5	Versione 64omp: baseline C + OpenMP	14
2.7.6	Versione 64omp: ASM AVX + OpenMP	14
2.8	Test di robustezza: edge cases	14
2.9	Benchmark prestazionali	15
3	Implementazione in linguaggio C	16
3.1	Strutture dati comuni e parametri	16
3.1.1	Allineamento in memoria e layout dei dati	17
3.2	Pipeline comune: fit e predict	17

3.2.1	Dual code-path: selettori C/ASM	17
3.3	Versione 32-bit (float)	18
3.3.1	Quantizzazione top- x	18
3.3.2	Distanza approssimata e lower bound	19
3.3.3	Fasi fit e predict	20
3.4	Versione 64-bit (double)	20
3.4.1	Motivazione: unrolling e prefetch	21
3.5	Versione 64-bit OpenMP (64omp)	21
3.5.1	Parallelizzazione nel fit	21
3.5.2	Parallelizzazione nel predict (per query)	21
3.5.3	Aggiornamento del peggiore tra i K candidati	22
3.6	Riepilogo delle differenze tra 32, 64 e 64omp	23
4	Ottimizzazioni in Assembly	24
4.1	Interfaccia, ABI e prerequisiti hardware	24
4.1.1	ABI effettiva (coerenza C/NASM)	24
4.1.2	Allineamento e sicurezza dei load	24
4.1.3	Perché queste tre routine	25
4.2	SSE su float (variante 32)	25
4.2.1	<code>approx_distance_asm</code> : quattro dot-product in parallelo	25
4.2.2	<code>euclidean_distance_asm</code> : differenza-quadrato-somma + radice	26
4.2.3	<code>compute_lower_bound_asm</code> : valore assoluto e massimo vettoriale	26
4.3	AVX su double (varianti 64 e 64omp)	26
4.3.1	Unrolling e FMA nelle routine AVX	27
4.4	Thread-safety nella variante 64omp	27
5	Compare results e criteri di validazione	28
5.1	Output richiesti	28
5.2	Formato <code>.ds2</code> e vincoli di compatibilità	28
5.2.1	Nota critica: salvataggio ID a 4 byte in 64 e 64omp	29
5.3	Cosa controlla il confronto	29
5.4	Criteri di uguaglianza e tolleranze	29
5.5	Workflow di confronto e significato di PASS	30
6	Test, validazione ed edge cases	31
6.1	Obiettivo dei test e criteri di robustezza	31
6.2	Assunzioni sui parametri e casi limite realistici	31
6.2.1	Dimensioni non allineate alla SIMD (gestione del tail)	32
6.2.2	Allineamento dei buffer e robustezza del wrapper	32
6.2.3	Robustezza multi-thread (versione 64omp)	32
6.2.4	Dataset di piccole dimensioni per validazione funzionale	32

6.3	Esiti dei test su dimensioni non allineate alla SIMD	33
6.4	Esiti dei test di robustezza multi-thread (64omp)	33
6.5	Coerenza tra versioni	33
7	Benchmark e benchmark_dimensioni	34
7.1	Metodologia sperimentale	34
7.2	Benchmark “compare” sulle istanze del docente	34
7.2.1	Cosa misuriamo	35
7.3	Benchmark_dimensioni: scalabilità con N e D	35
7.3.1	Aspettative teoriche (coerenti con l’implementazione)	35
7.4	Interpretazione dei risultati	36
7.4.1	Come riportiamo i risultati in relazione	36
7.5	Discussione dei risultati	36

Capitolo 1

Introduzione e richieste del progetto

Il problema della ricerca dei *K-Nearest Neighbors* (K-NN) è un metodo fondamentale nell'analisi dei dati e nel machine learning. Dato un insieme di punti immersi in uno spazio metrico e un punto di query, l'obiettivo è individuare i K punti del dataset più vicini alla query secondo una funzione di distanza. Nonostante la semplicità concettuale, l'approccio esatto richiede il calcolo della distanza tra la query e tutti i punti del dataset, con costo $O(n \cdot D)$ per query: tale costo diventa rapidamente proibitivo per dataset molto grandi e/o spazi ad alta dimensionalità.

Il progetto del corso di *Architetture Avanzate dei Sistemi di Elaborazione* richiede l'implementazione di un algoritmo di ricerca *approssimata* dei K vicini, in cui si riducono drasticamente i tempi di esecuzione accettando una perdita controllata di accuratezza. L'obiettivo principale non è soltanto la correttezza funzionale, ma soprattutto l'ottimizzazione delle prestazioni sfruttando in modo esplicito le caratteristiche hardware (SIMD e multicore).

1.1 Richieste della traccia e vincoli di consegna

La traccia fornita dal docente impone vincoli precisi sia a livello di algoritmo sia a livello di struttura del progetto e modalità di esecuzione. In particolare:

- **Implementazione di base in C:** sviluppo della logica principale dell'algoritmo, rispettando lo scheletro e le interfacce fornite.
- **Tre versioni del progetto:**
 - **32-bit** in singola precisione (`float`) con ottimizzazioni **SSE**;
 - **64-bit** in doppia precisione (`double`) con ottimizzazioni **AVX**;
 - **64-bit OpenMP** (`double`) con **AVX** e parallelizzazione **OpenMP**.
- **Funzioni critiche ottimizzate:** riscrittura in assembly (dove richiesto) delle routine più costose, tipicamente: quantizzazione, distanza approssimata, limite inferiore (lower bound) e/o distanza euclidea.

- **Interfaccia e formato dati:** utilizzo del formato dataset/query previsto (file `.ds2`) e produzione degli output nel formato atteso (ID e distanze dei vicini).
- **Struttura e script:** lo sviluppo deve avvenire *senza alterare la struttura richiesta* (directory, header, script), riempiendo i punti lasciati incompleti.

1.2 Panoramica dell'algoritmo (QuantPivot)

L'algoritmo implementato si basa su due idee chiave:

1. **Quantizzazione dei vettori** per ridurre il costo delle operazioni di distanza;
2. **Indicizzazione tramite pivot e pruning** per evitare molti confronti durante la fase di ricerca.

Le ottimizzazioni SIMD/OpenMP introdotte nelle versioni 32/64/64omp non modificano la logica complessiva dell'algoritmo, ma agiscono sul modo in cui le singole operazioni vengono eseguite.

1.2.1 Il problema dei K-Nearest Neighbors

Dato un dataset $DS = \{v_1, v_2, \dots, v_n\}$ con $v_i \in \mathbb{R}^D$ e una query $q \in \mathbb{R}^D$, il K-NN consiste nel trovare l'insieme $N \subseteq DS$ di cardinalità K tale che:

$$\forall v \in N, \forall w \in DS \setminus N : d(q, v) \leq d(q, w),$$

dove $d(\cdot, \cdot)$ è la distanza euclidea:

$$d(v, w) = \sqrt{\sum_{i=1}^D (v_i - w_i)^2}.$$

1.2.2 Quantizzazione dei vettori (top- x sparsa)

Per ridurre il costo del calcolo delle distanze, l'algoritmo rappresenta ogni vettore $v \in \mathbb{R}^D$ tramite due vettori sparsi v^+ e v^- , costruiti selezionando le x componenti di v con valore assoluto maggiore (top- x). Sia $I_x(v)$ l'insieme degli indici corrispondenti alle x componenti a massima magnitudine. Allora, per ogni $i \in \{1, \dots, D\}$:

$$(v^+)_i = \begin{cases} 1 & \text{se } i \in I_x(v) \text{ e } v_i \geq 0, \\ 0 & \text{altrimenti,} \end{cases} \quad (v^-)_i = \begin{cases} 1 & \text{se } i \in I_x(v) \text{ e } v_i < 0, \\ 0 & \text{altrimenti.} \end{cases}$$

In questo modo ciascun vettore quantizzato contiene esattamente x valori non nulli (distribuiti tra v^+ e v^-) e cattura sia l'informazione di *selezione* delle componenti dominanti sia il loro *segno*. La realizzazione efficiente della selezione top- x e la costruzione dei vettori sparsi vengono discusse nei capitoli successivi.

1.2.3 Distanza approssimata

A partire dai vettori quantizzati, la distanza approssimata tra due punti v e w è espressa tramite prodotti scalari:

$$\tilde{d}(v, w) = \langle v^+, w^+ \rangle + \langle v^-, w^- \rangle - \langle v^+, w^- \rangle - \langle v^-, w^+ \rangle.$$

Questa forma è favorevole all'uso di SSE/AVX poiché riconduce il calcolo a somme e moltiplicazioni elementari, facilmente vettorizzabili.

1.2.4 Indicizzazione tramite pivot (fase fit)

Si seleziona un insieme di h pivot $P = \{p_0, \dots, p_{h-1}\} \subset DS$ in modo deterministico, con passo $\lfloor n/h \rfloor$. Durante la fase di **fit**, per ogni punto v del dataset e per ogni pivot p_j si calcola e memorizza:

$$\text{index}[v, j] = \tilde{d}(v, p_j).$$

Questa struttura consente di velocizzare la fase di ricerca riducendo il numero di candidati effettivamente valutati con distanze più costose.

1.2.5 Querying e pruning (fase predict)

Data una query q , si quantizza q e si calcola $\tilde{d}(q, p_j)$ per tutti i pivot. Per ciascun punto v si valuta un limite inferiore:

$$LB(q, v) = \max_{0 \leq j < h} \left| \tilde{d}(v, p_j) - \tilde{d}(q, p_j) \right|.$$

Se $LB(q, v)$ è già maggiore della distanza del peggior tra i K candidati correnti, il punto v viene scartato: non può entrare nei K migliori senza violare il bound. Solo sui candidati finali si calcola la distanza euclidea esatta per produrre gli output definitivi.

1.3 Struttura del progetto

Lo scheletro fornito dal docente impone la presenza di tre versioni distinte, organizzate nelle directory `src/32`, `src/64` e `src/64omp`. A livello di repository sono presenti anche

script Python per eseguire test e benchmark, oltre al package Python che espone le tre implementazioni.

```
|_ README.md
|_ test.py
|_ benchmark.py
|_ benchmark_dimensioni.py
|_ ProgettoGruppo6/
|   |_ README.md
|   |_ pyproject.toml
|   |_ setup.py
|   |_ gruppo6/
|       |_ __init__.py
|       |_ quantpivot32/
|           |_ __init__.py
|       |_ quantpivot64/
|           |_ __init__.py
|       |_ quantpivot64omp/
|           |_ __init__.py
|   |_ src/
|       |_ 32/
|           |_ common.h
|           |_ main.c
|           |_ quantpivot32.c
|           |_ quantpivot32_py.c
|           |_ quantpivot32.nasm
|           |_ sseutils32.nasm
|           |_ sseutils32.o
|       |_ 64/
|           |_ common.h
|           |_ main.c
|           |_ quantpivot64.c
|           |_ quantpivot64_py.c
|           |_ quantpivot64.nasm
|           |_ sseutils64.nasm
|           |_ sseutils64.o
|       |_ 64omp/
|           |_ common.h
|           |_ main.c
|           |_ quantpivot64omp.c
|           |_ quantpivot64omp_py.c
|           |_ quantpivot64omp.nasm
|           |_ sseutils64.nasm
|           |_ sseutils64.o
```

Le tre sottodirectory in `src/` contengono ciascuna: un `common.h` (definizioni di tipo numerico e allineamento), il sorgente C principale dell'algoritmo (`quantpivot*.c`), il wrapper Python/C (`quantpivot*_py.c`) e le routine NASM ottimizzate. I file `main.c` sono quelli

forniti dal docente e vengono mantenuti invariati; l'esecuzione end-to-end avviene tramite gli script `test.py` e i benchmark. Nel repository sono inoltre presenti file `.ds2` di dataset/query e, quando previsti dagli script di test, file di output attesi utilizzati per il confronto dei risultati.

Capitolo 2

Compilazione, esecuzione e validazione sperimentale

Questo capitolo descrive come compilare ed eseguire il progetto nelle diverse configurazioni, e come validare la correttezza dei risultati. Per mantenere la relazione riproducibile su qualunque macchina (e con qualunque percorso di installazione), i comandi sono presentati **senza path assoluti** e si assumono eseguiti dalla **root del repository**, con un virtual environment Python attivo.

2.1 Dipendenze e ambiente di esecuzione

Gli script di test/benchmark e i wrapper Python richiedono un ambiente Python 3 con librerie numeriche. In particolare:

- **NumPy**: gestione degli array e interfacciamento con l'estensione C;
- **pyFFTW**: usato per allocare array *allineati* in memoria (vincolo richiesto dai wrapper per SSE/AVX);
- **NASM**: necessario per assemblare i kernel ottimizzati quando l'ASM è abilitato;
- **FFTW3** (di sistema): dipendenza di **pyFFTW**.

```
# Esempio su sistemi Debian/Ubuntu
```

```
sudo apt update
```

```
sudo apt install python3-venv nasm libfftw3-dev
```

```
# Virtual environment
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
# Dipendenze Python
```

```
pip install -U pip setuptools wheel
pip install numpy pyfftw
```

2.2 Installazione editable del package

Per eseguire i test dalla root del repository, il package viene installato in modalità editable:

```
source venv/bin/activate
pip install -e .
```

2.3 Integrazione Python/C: wrapper e vincoli di allineamento

Il progetto espone le funzioni `fit` e `predict` come estensioni Python, tramite wrapper dedicati (`quantpivot32_py.c`, `quantpivot64_py.c`, `quantpivot64omp_py.c`). Questa scelta consente di usare gli script `test.py` e `benchmark.py` come entrypoint unificati, indipendentemente dalla variante (32/64/64omp) e dalla modalità (C o ASM).

Un punto cruciale è che i wrapper impongono vincoli su: (i) **tipo numerico** degli array NumPy (`float32` per 32-bit, `float64` per 64/64omp), (ii) **allineamento in memoria** coerente con le istruzioni SIMD usate (16 byte per SSE, 32 byte per AVX). L'allineamento è necessario per evitare penalità di prestazioni e, in presenza di load/store allineati in assembly, per prevenire comportamenti indefiniti.

2.3.1 Check di allineamento nel wrapper

Nel wrapper, dopo aver ottenuto il puntatore ai dati tramite NumPy, viene verificato che l'indirizzo sia multiplo della costante `align` definita in `common.h`:

```
type* dataset = (type*)PyArray_DATA((PyArrayObject*)ds_array);

uintptr_t addr = (uintptr_t)dataset;
int is_aligned = (addr % align == 0);

if (!is_aligned){
    PyErr_SetString(PyExc_ValueError, "Input array (DS) not aligned");
    return NULL;
}
```

In caso di input non allineato, l'esecuzione viene interrotta con un errore esplicito, evitando che la parte C/ASM operi su buffer non conformi ai requisiti SIMD.

2.4 Formato .ds2 e caricamento in memoria (script di test)

Dataset e query sono forniti in formato binario `.ds2`, composto da:

- **header** con due interi a 32 bit: numero di righe n e numero di colonne d ;
- **payload** con i valori in formato floating point, memorizzati in ordine row-major.

Gli script di test caricano i file `.ds2` e allocano gli array con **allineamento esplicito** tramite `pyfftw.empty_aligned`, così da rispettare i vincoli imposti dai wrapper:

```
def load_ds2(filename, dtype, alignment):  
    with open(filename, 'rb') as f:  
        header = f.read(8)  
        n, d = struct.unpack('ii', header)  
        aligned = pyfftw.empty_aligned((n, d), dtype=dtype, n=alignment)  
        f.readinto(aligned.data)  
    return aligned
```

In questo modo la pipeline di test garantisce che gli input passati alle estensioni C siano coerenti con la versione selezionata (32/64/64omp) e con i relativi requisiti di allineamento.

2.5 Pulizia degli artefatti di build

Prima di passare da una configurazione all'altra (ad esempio da C a ASM o tra versioni diverse), è consigliato rimuovere gli artefatti della build precedente per evitare il riutilizzo di file obsoleti (`build/`, `.so`, `.o`).

Nei comandi riportati nelle sezioni successive, la pulizia viene eseguita esplicitamente prima di ogni compilazione.

2.6 Modalità di compilazione: baseline C e ASM

Il progetto supporta due modalità:

- **Baseline C**: kernel NASM disattivati (default);
- **ASM attivo**: kernel NASM SSE/AVX abilitati *a build-time* tramite variabili d'ambiente.

La logica dell'algoritmo e il formato degli output restano invariati: cambia solo quale implementazione viene invocata nelle funzioni critiche (approx distance, lower bound, euclidean distance).

2.6.1 Abilitazione ASM a build-time

L'abilitazione delle routine assembly avviene impostando una variabile d'ambiente prima della compilazione:

- `USE_ASM_32=1` per la versione 32-bit (SSE);
- `USE_ASM_64=1` per la versione 64-bit (AVX);
- `USE_ASM_OMP=1` per la versione 64omp (AVX + OpenMP).

Quando attive, queste variabili fanno sì che `setup.py` aggiunga le macro `USE_ASM_*` ai flag di compilazione e colleghi gli oggetti generati da NASM.

2.7 Esecuzione e validazione: `test standard + compare results`

Il flusso standard di validazione è:

1. compilazione dell'estensione (`python3 setup.py build_ext --inplace`);
2. esecuzione di `test.py` su dataset/query forniti;
3. confronto degli output con `compare_results.py`. Lo script `compare_results.py` verifica che: (i) gli ID dei vicini restituiti coincidano con quelli attesi, (ii) le distanze siano compatibili entro tolleranza numerica (necessaria per differenze di precisione e ordine di somma).

2.7.1 Versione 32-bit: baseline C

```
rm -rf build/ gruppo6/quantpivot32/*.so src/32/*.o
python3 setup.py build_ext --inplace
python3 test.py dataset_2000x256_32.ds2 query_2000x256_32.ds2 16 8 64 32
python3 compare_results.py
```

2.7.2 Versione 32-bit: ASM SSE

```
rm -rf build/ gruppo6/quantpivot32/*.so src/32/*.o
USE_ASM_32=1 python3 setup.py build_ext --inplace
python3 test.py dataset_2000x256_32.ds2 query_2000x256_32.ds2 16 8 64 32
python3 compare_results.py
```

2.7.3 Versione 64-bit: baseline C

```
rm -rf build/ gruppo6/quantpivot64/*.so src/64/*.o
python3 setup.py build_ext --inplace
python3 test.py dataset_2000x256_64.ds2 query_2000x256_64.ds2 16 8 64 64
python3 compare_results.py
```

2.7.4 Versione 64-bit: ASM AVX

```
rm -rf build/ gruppo6/quantpivot64/*.so src/64/*.o
USE_ASM_64=1 python3 setup.py build_ext --inplace
python3 test.py dataset_2000x256_64.ds2 query_2000x256_64.ds2 16 8 64 64
python3 compare_results.py
```

2.7.5 Versione 64omp: baseline C + OpenMP

```
rm -rf build/ gruppo6/quantpivot64omp/*.so src/64omp/*.o
python3 setup.py build_ext --inplace
python3 test.py dataset_2000x256_64.ds2 query_2000x256_64.ds2 16 8 64
64omp
python3 compare_results.py
```

2.7.6 Versione 64omp: ASM AVX + OpenMP

```
rm -rf build/ gruppo6/quantpivot64omp/*.so src/64omp/*.o
USE_ASM_OMP=1 python3 setup.py build_ext --inplace
python3 test.py dataset_2000x256_64.ds2 query_2000x256_64.ds2 16 8 64
64omp
python3 compare_results.py
```

2.8 Test di robustezza: edge cases

Oltre ai test standard, il progetto include `test_edge_cases.py`, che verifica la correttezza su dimensioni non ideali (dispari e non multiple della granularità SIMD) e su dataset più grandi. Questi test sono importanti perché stressano la gestione della *tail* nei loop vettorializzati e verificano l'assenza di errori ai bordi (off-by-one, accessi fuori limite, gestione corretta delle ultime iterazioni).

```
source venv/bin/activate
pip install -e .
python3 test_edge_cases.py
```

2.9 Benchmark prestazionali

Per valutare i tempi di esecuzione e confrontare le configurazioni (C vs ASM, e scaling con dimensioni diverse), sono disponibili due script:

- `benchmark.py`: confronto prestazionale nelle configurazioni previste;
- `benchmark_dimensioni.py`: analisi di scalabilità al variare di N e/o D (e dei parametri dell'algoritmo).

```
python3 benchmark.py
python3 benchmark_dimensioni.py
```

Capitolo 3

Implementazione in linguaggio C

In questo capitolo descriviamo l'implementazione dell'algoritmo QuantPivot in linguaggio C così come realizzata nel progetto. La traccia prevede tre varianti che condividono la stessa pipeline logica ma differiscono per: (i) tipo numerico (**float** vs **double**), (ii) requisiti di allineamento (16 vs 32 byte), (iii) ottimizzazioni ingegneristiche nello scanning (*unrolling/prefetch*) e, nella variante **64omp**, parallelizzazione OpenMP.

3.1 Strutture dati comuni e parametri

Ogni versione definisce tipo e allineamento in `common.h`. Ad esempio:

```
// 32-bit
#define type    float
#define align   16

// 64-bit e 64omp
#define type    double
#define align   32
```

La struttura `params` raccoglie dataset e query, indici dei pivot, strutture quantizzate e output:

```
typedef struct{
    MATRIX DS;           // dataset (N x D)
    MATRIX Q;           // query (nq x D)
    int* P;             // indici dei pivot (h)
    MATRIX index;        // indice (N x h)
    MATRIX ds_plus;      // dataset quantizzato positivo (N x D)
    MATRIX ds_minus;     // dataset quantizzato negativo (N x D)
    int* id_nn;          // output ID (nq x k)
    MATRIX dist_nn;      // output distanze (nq x k)
    int h, k, x;         // #pivot, #vicini, quantizzazione top-x
    int N, D, nq;        // dimensioni dataset/query
}
```



```

    int silent;
    bool first_fit_call;
} params;

```

3.1.1 Allineamento in memoria e layout dei dati

Le tre varianti differiscono anche per l'allineamento richiesto, coerente con le estensioni SIMD:

- **32-bit (SSE)**: allineamento a 16 byte (registri XMM da 128 bit);
- **64-bit e 64omp (AVX)**: allineamento a 32 byte (registri YMM da 256 bit).

Le principali strutture dati (dataset, query, vettori quantizzati, indice) vengono allocate tramite `_mm_malloc` specificando l'allineamento desiderato. L'accesso allineato migliora l'efficienza dei load/store vettoriali e, soprattutto, evita comportamenti indefiniti quando l'assembly utilizza istruzioni che assumono un allineamento specifico.

3.2 Pipeline comune: fit e predict

La pipeline logica è identica nelle tre versioni:

- **Fit**: selezione dei pivot, quantizzazione del dataset, costruzione dell'indice `index` con distanze approssimate punto-pivot;
- **Predict**: quantizzazione delle query, calcolo distanze query-pivot, pruning tramite lower bound, selezione candidati e raffinamento finale tramite distanza euclidea.

3.2.1 Dual code-path: selettori C/ASM

Le routine più costose (distanza approssimata, lower bound, distanza euclidea) sono incapsulate in funzioni `*_sel` che invocano l'implementazione disponibile (C oppure ASM quando abilitata a build-time). In forma schematica:

```

static inline type approx_distance_sel(
    const type* vplus, const type* vminus,
    const type* wplus, const type* wminus, int D)
{
#ifdef USE_ASM_APPROX
    return approx_distance_asm(vplus, vminus, wplus, wminus, D);
#else
    return approx_distance(vplus, vminus, wplus, wminus, D);
#endif
}

```

```
#endif
}
```

Lo stesso pattern è adottato anche per `compute_lower_bound_sel` e `euclidean_distance_sel`.

3.3 Versione 32-bit (float)

3.3.1 Quantizzazione top- x

La quantizzazione trasforma ogni vettore v in due vettori binari v^+ e v^- , rappresentati come array di lunghezza D con valori in $\{0,1\}$. La sparsità deriva dal fatto che solo x componenti risultano non nulle (top- x). Nel codice, l'implementazione: (i) azzerà gli output, (ii) calcola gli assoluti e mantiene un array di indici, (iii) usa *Quickselect* per portare i top- x in testa, (iv) ordina solo i primi x elementi (insertion sort) poiché $x \ll D$.

```
static inline void quantize_vector(const type* v,
                                   type* vplus,
                                   type* vminus,
                                   int x,
                                   int D,
                                   int* scratch_indices,
                                   type* scratch_abs_vals)
{
    memset(vplus, 0, (size_t)D * sizeof(type));
    memset(vminus, 0, (size_t)D * sizeof(type));

    if (x <= 0) return;
    if (x > D) x = D;

    for (int i = 0; i < D; i++) {
        scratch_indices[i] = i;
        scratch_abs_vals[i] = fabsf(v[i]);
    }

    quickselect_top_x(scratch_abs_vals, scratch_indices, 0, D - 1, x);

    for (int j = 1; j < x; j++) {
        type key_val = scratch_abs_vals[j];
        int key_idx = scratch_indices[j];
        int k = j - 1;
        while (k >= 0 && scratch_abs_vals[k] < key_val) {
```

```

        scratch_abs_vals[k + 1] = scratch_abs_vals[k];
        scratch_indices[k + 1] = scratch_indices[k];
        k--;
    }
    scratch_abs_vals[k + 1] = key_val;
    scratch_indices[k + 1] = key_idx;
}

for (int count = 0; count < x; count++) {
    int idx = scratch_indices[count];
    if (v[idx] >= 0.0f) vplus[idx] = 1.0f;
    else                vminus[idx] = 1.0f;
}
}

```

3.3.2 Distanza approssimata e lower bound

La distanza approssimata tra due vettori quantizzati è la combinazione di quattro prodotti scalari (pp, mm, pm, mp):

```

static inline type approx_distance(const type* vplus, const type* vminus,
                                   const type* wplus, const type* wminus,
                                   int D)
{
    type dot_pp = 0.0f, dot_mm = 0.0f, dot_pm = 0.0f, dot_mp = 0.0f;
    for (int i = 0; i < D; i++) {
        dot_pp += vplus[i] * wplus[i];
        dot_mm += vminus[i] * wminus[i];
        dot_pm += vplus[i] * wminus[i];
        dot_mp += vminus[i] * wplus[i];
    }
    return dot_pp + dot_mm - dot_pm - dot_mp;
}

```

Il pruning usa il lower bound come massimo scostamento sulle distanze dai pivot:

```

static inline type compute_lower_bound(const type* idx_v,
                                       const type* qpivot,
                                       int h)
{
    type max_lb = 0.0f;
    for (int j = 0; j < h; j++) {
        type diff = fabsf(idx_v[j] - qpivot[j]);
    }
}

```

```

        if (diff > max_lb) max_lb = diff;
    }
    return max_lb;
}

```

Nelle versioni a doppia precisione la stessa routine utilizza **fabs** al posto di **fabsf**.

3.3.3 Fasi fit e predict

La fase **fit** seleziona i pivot con passo N/h , quantizza il dataset e costruisce **index** con distanze approssimate punto-pivot. La fase **predict** quantizza la query, calcola **qpivot**, esegue lo scanning del dataset con pruning tramite lower bound e raffina i migliori candidati con distanza euclidea.

Nota implementativa: nella versione 32 l'inizializzazione della variabile **worst_dist** usa **FLT_MAX**; nelle versioni 64/64omp l'equivalente naturale è **DBL_MAX** (da **float.h**).

3.4 Versione 64-bit (double)

La versione 64-bit mantiene la stessa pipeline ma usa **double** e allineamento a 32 byte. Rispetto alla 32, lo scanning in **predict** introduce ottimizzazioni ingegneristiche mirate a ridurre la latenza di memoria:

- **prefetch iniziale** delle prime iterazioni;
- **unrolling manuale x4** sul ciclo dei candidati;
- **prefetch continuo** a distanza fissa per mantenere la pipeline piena.

```

// Batch prefetch aggressivo per le prime 64 iterazioni
for (int pf = 0; pf < 64 && pf < N; pf++) {
    __builtin_prefetch(&input->index[(size_t)pf * (size_t)h], 0, 0);
    __builtin_prefetch(&input->ds_plus[(size_t)pf * (size_t)D], 0, 0);
    __builtin_prefetch(&input->ds_minus[(size_t)pf * (size_t)D], 0, 0);
}

// Loop unrolled x4
for (; v <= N - 4; v += 4) {
    if (v + 64 < N) {
        __builtin_prefetch(&input->index[(size_t)(v+64) * (size_t)h],
            0, 0);
        __builtin_prefetch(&input->ds_plus[(size_t)(v+64) * (size_t)D],
            0, 0);
    }
}

```

```

        __builtin_prefetch(&input->ds_minus[(size_t)(v+64) * (size_t)D],
        0, 0);
    }

    // candidato v+0, v+1, v+2, v+3 (stesso schema)
    // LB = compute_lower_bound_sel(...)
    // d = approx_distance_sel(...)
    // aggiornamento worst in O(k)
}

```

3.4.1 Motivazione: unrolling e prefetch

Nello scanning del dataset, il costo non è dato solo dalle operazioni aritmetiche ma anche dall'accesso a strutture grandi (index e vettori quantizzati), che può causare cache miss. Il loop unrolled riduce overhead e aumenta ILP, mentre il prefetch anticipa il caricamento dei dati usati nelle iterazioni successive, nascondendo parte della latenza di memoria.

3.5 Versione 64-bit OpenMP (64omp)

La versione 64omp introduce parallelismo a livello di thread mantenendo invariata la logica dell'algoritmo. La scelta progettuale è parallelizzare dove le iterazioni sono indipendenti, minimizzando sincronizzazione e contese.

3.5.1 Parallelizzazione nel fit

Nel `fit` vengono parallelizzate operazioni naturalmente indipendenti:

- **quantizzazione del dataset:** ogni vettore del dataset può essere quantizzato indipendentemente;
- **costruzione dell'indice:** le righe di `index` sono indipendenti (una riga per punto del dataset).

Per evitare race condition, gli scratch buffer necessari alla quantizzazione (indici e valori assoluti) sono allocati **per-thread**.

3.5.2 Parallelizzazione nel predict (per query)

Nel `predict` la parallelizzazione avviene **per query**:

- **quantizzazione delle query** distribuita tra thread;

- **ricerca K-NN per query** con `schedule(dynamic,10)` per bilanciare carichi non uniformi.

Ogni thread elabora un sottoinsieme di query e, per ogni query, scansiona il dataset mantenendo pruning e ottimizzazioni (prefetch/unrolling). Le strutture temporanee (**knn**, **qpivot** e scratch della quantizzazione) sono allocate **localmente al thread**, evitando sincronizzazione.

```
{
    neighbor* knn = (neighbor*)_mm_malloc(k * sizeof(neighbor), 32);
    type* qpivot = (type*)_mm_malloc(h * sizeof(type), 32);

    #pragma omp for schedule(dynamic, 10)
    for(int q = 0; q < nq; q++){
        // init knn
        // compute qpivot
        // scan DS con pruning
        // raffinamento euclideo e scrittura output
    }

    _mm_free(qpivot);
    _mm_free(knn);
}
```

3.5.3 Aggiornamento del peggiore tra i K candidati

Durante lo scanning, quando un nuovo candidato entra nei **k** migliori, viene aggiornato il “peggiore” (*worst*) tra i K. Per ridurre l’overhead, è presente una routine dedicata ottimizzata per il caso frequente $k = 8$:

```
static inline void update_knn_fast(neighbor* knn, int k, int id,
                                   type dist, type* worst_dist,
                                   int* worst_idx)
{
    knn[*worst_idx].id = id;
    knn[*worst_idx].dist = dist;

    *worst_dist = knn[0].dist;
    *worst_idx = 0;

    if (k >= 8) {
        if (knn[1].dist > *worst_dist) { *worst_dist = knn[1].dist;
```

```

    *worst_idx = 1; }
    if (knn[2].dist > *worst_dist) { *worst_dist = knn[2].dist;
    *worst_idx = 2; }
    if (knn[3].dist > *worst_dist) { *worst_dist = knn[3].dist;
    *worst_idx = 3; }
    if (knn[4].dist > *worst_dist) { *worst_dist = knn[4].dist;
    *worst_idx = 4; }
    if (knn[5].dist > *worst_dist) { *worst_dist = knn[5].dist;
    *worst_idx = 5; }
    if (knn[6].dist > *worst_dist) { *worst_dist = knn[6].dist;
    *worst_idx = 6; }
    if (knn[7].dist > *worst_dist) { *worst_dist = knn[7].dist;
    *worst_idx = 7; }
    return;
}

for (int i = 1; i < k; i++) {
    if (knn[i].dist > *worst_dist) {
        *worst_dist = knn[i].dist;
        *worst_idx = i;
    }
}
}

```

3.6 Riepilogo delle differenze tra 32, 64 e 64omp

A parità di pipeline algoritmica, le tre varianti si distinguono per scelte implementative orientate alla piattaforma:

- **32:** `float`, allineamento 16 byte, baseline pensata per SSE; maggiore sensibilità numerica ma minore footprint di memoria.
- **64:** `double`, allineamento 32 byte, ottimizzazioni di scanning (prefetch/unrolling) per ridurre latenza di memoria e overhead di loop.
- **64omp:** `double`, allineamento 32 byte, parallelizzazione OpenMP per query e per fasi indipendenti del fit, con strutture temporanee per-thread per evitare race condition.

I dettagli sulle ottimizzazioni NASM (SSE/AVX) sono discussi nel Capitolo dedicato all'assembly, mentre formato di output e criteri di confronto sono trattati nel capitolo di validazione.

Capitolo 4

Ottimizzazioni in Assembly

Questo capitolo descrive le ottimizzazioni introdotte tramite routine in assembly NASM, utilizzate per accelerare le parti computazionalmente dominanti dell'algoritmo QuantPivot. Le routine riscritte sono tre, richiamate (quando abilitate) dalle varianti 32, 64 e 64omp:

- `approx_distance_asm`: distanza approssimata tra vettori quantizzati;
- `euclidean_distance_asm`: distanza euclidea per il raffinamento finale;
- `compute_lower_bound_asm`: lower bound sui pivot per il pruning.

L'assembly viene abilitato a build-time tramite variabili d'ambiente impostate prima della compilazione: `USE_ASM_32=1`, `USE_ASM_64=1`, `USE_ASM_OMP=1`. Quando attive, `setup.py` definisce le macro `USE_ASM_*` e collega gli oggetti prodotti da NASM; nel codice C, tramite funzioni selettori (`*_sel`), le chiamate vengono instradate alle versioni assembly.

4.1 Interfaccia, ABI e prerequisiti hardware

4.1.1 ABI effettiva (coerenza C/NASM)

Le tre routine NASM sono collegate come oggetti `elf64` e vengono chiamate da C tramite firme coerenti. Di conseguenza, le routine seguono la convenzione di chiamata **System V AMD64**: i puntatori ai vettori arrivano in registri (tipicamente `RDI`, `RSI`, `RDX`, `RCX`), mentre i parametri interi (come `D` o `h`) arrivano in un registro dedicato (es. `R8D`). Il valore di ritorno è prodotto in un registro SIMD (parte bassa di `XMM0`) e poi interpretato come scalare.

4.1.2 Allineamento e sicurezza dei load

Le routine assumono che i buffer principali siano allocati con l'allineamento previsto dal progetto:

- 32: float e allineamento a 16 byte (SSE);
- 64/64omp: double e allineamento a 32 byte (AVX).

Nei loop principali vengono privilegiati load allineati (massimo throughput), mentre la parte finale gestisce i residui (*tail*) anche con load non allineati per evitare letture fuori limite quando la dimensione non è multipla del vettore SIMD.

4.1.3 Perché queste tre routine

Sono le funzioni più invocate e più costose:

- `approx_distance`: chiamata in `fit` per costruire `index` e in `predict` nello scanning dei candidati;
- `compute_lower_bound`: chiamata per ogni candidato durante il pruning;
- `euclidean_distance`: chiamata sui candidati finali per produrre le distanze definitive.

Ottimizzarle significa ridurre direttamente il tempo dominante sia in `fit` sia in `predict`.

4.2 SSE su float (variante 32)

Nella variante 32 si lavora in singola precisione (`float`) usando registri `XMM` da 128 bit, che contengono 4 float. L'obiettivo è ridurre di circa un fattore 4 le iterazioni dei loop su D e su h , sfruttando parallelismo SIMD e riducendo overhead di loop.

4.2.1 `approx_distance_asm`: quattro dot-product in parallelo

La distanza approssimata è:

$$\tilde{d}(v, w) = \langle v^+, w^+ \rangle + \langle v^-, w^- \rangle - \langle v^+, w^- \rangle - \langle v^-, w^+ \rangle.$$

L'implementazione SSE:

- usa **più accumulatori SIMD** (uno per ciascun termine) per ridurre dipendenze tra istruzioni e aumentare ILP;
- esegue un loop vettoriale su blocchi da 4 componenti: `load` $\rightarrow$ `multiply` $\rightarrow$ `accumulate`;
- applica una **riduzione orizzontale** (somma delle 4 lane) per ottenere uno scalare;
- gestisce l'eventuale **tail** con un breve loop scalare sulle componenti residue.

In pratica, la stessa operazione del C viene svolta con meno iterazioni e con unità vettoriali sempre occupate.

4.2.2 `euclidean_distance_asm`: differenza-quadrato-somma + radice

La distanza euclidea:

$$d(v, w) = \sqrt{\sum_{i=1}^D (v_i - w_i)^2}$$

viene calcolata con il pattern SIMD classico:

- load vettoriale di 4 float da `v` e `w`;
- sottrazione vettoriale, quadrato (moltiplicazione vettoriale) e accumulo;
- riduzione orizzontale della somma;
- `sqrt` finale sullo scalare.

Anche qui il tail (se $D \bmod 4 \neq 0$) viene gestito scalarmente.

4.2.3 `compute_lower_bound_asm`: valore assoluto e massimo vettoriale

Il lower bound è:

$$LB(q, v) = \max_{0 \leq j < h} |\text{index}[v, j] - qpivot[j]|.$$

L'ottimizzazione SSE si basa su:

- sottrazione vettoriale tra blocchi di 4 componenti;
- calcolo del valore assoluto tramite maschera sul bit di segno;
- massimo vettoriale (`max`) mantenendo un accumulatore di massimo;
- riduzione finale a massimo scalare e tail scalare se h non è multiplo di 4.

4.3 AVX su double (varianti 64 e 64omp)

Nelle varianti `64/64omp` si lavora in doppia precisione (`double`) usando registri YMM da 256 bit, che contengono 4 double. Rispetto alla versione SSE, la struttura delle routine è la stessa, ma:

- si sfruttano istruzioni AVX su 256 bit per processare 4 double per iterazione;
- nei loop principali si usano load allineati a 32 byte quando possibile;
- la riduzione finale combina prima le due metà da 128 bit e poi completa la riduzione a scalare;
- viene eseguita `vzeroupper` al termine delle routine per evitare penalità di transizione AVX→SSE nel chiamante.

4.3.1 Unrolling e FMA nelle routine AVX

Le routine AVX implementano anche ottimizzazioni di micro-architettura:

- **loop unrolling** (più blocchi SIMD per iterazione) per ridurre overhead di controllo e aumentare ILP;
- uso di **FMA** (fused multiply-add) dove disponibile: l'accumulo $a \leftarrow a + b \cdot c$ viene eseguito in un'unica istruzione, riducendo istruzioni totali e migliorando throughput.

La parte finale gestisce sempre i residui prima con un loop SIMD da 4 e poi con un loop scalare.

4.4 Thread-safety nella variante 64omp

Nella variante `64omp`, le routine NASM possono essere invocate simultaneamente da thread diversi. Sono **thread-safe** perché:

- operano su input in sola lettura (puntatori a buffer già allocati);
- usano esclusivamente registri e variabili locali sullo stack della funzione;
- non mantengono stato globale e non scrivono in strutture condivise.

Di conseguenza, la parallelizzazione OpenMP può richiamare le routine assembly senza sincronizzazione aggiuntiva: la sincronizzazione resta necessaria solo per la gestione delle strutture dati condivise a livello C (output e strutture globali), non per il calcolo SIMD interno.

Capitolo 5

Compare results e criteri di validazione

In questa sezione descriviamo la logica di verifica utilizzata per controllare la correttezza degli output prodotti dal progetto rispetto ai file attesi dal framework del docente. Poiché la valutazione avviene tramite esecuzione automatica, la correttezza non riguarda solo l'algoritmo, ma anche: (i) formato dei file, (ii) dimensioni e tipi dei dati scritti, (iii) coerenza tra versioni (32, 64, 64omp).

5.1 Output richiesti

Per ogni dataset di query, l'esecuzione produce due file principali:

- `out_idnn.ds2`: matrice degli ID dei K vicini, di dimensione $n_q \times K$;
- `out_distnn.ds2`: matrice delle distanze corrispondenti, di dimensione $n_q \times K$.

In generale, la pipeline del progetto calcola i candidati tramite pruning (lower bound su pivot) e poi raffina i migliori con distanza euclidea; di conseguenza, gli ID prodotti sono associati alle distanze finali calcolate in fase di refinement.

5.2 Formato .ds2 e vincoli di compatibilità

I file `.ds2` seguono una struttura semplice:

- **header** con due interi a 32 bit: `n` (numero righe) e `k` (numero colonne);
- **payload** contenente i dati in ordine row-major.

5.2.1 Nota critica: salvataggio ID a 4 byte in 64 e 64omp

Un vincolo essenziale (esplicitato dal docente) riguarda il salvataggio degli ID nella versione 64 e 64omp: gli ID devono essere salvati come `int` a 4 byte, non come interi a 8 byte. Per garantire piena compatibilità con lo script di confronto del docente, nelle versioni 64 e 64omp viene utilizzata la funzione:

```
save_int_data(filename, (int*)X, n, k)
```

in modo da forzare il formato richiesto in output.

5.3 Cosa controlla il confronto

La validazione viene eseguita verificando i seguenti aspetti:

1. **Coerenza del formato:** lettura corretta di header (`n`, `k`) e dimensioni attese.
2. **Tipo e layout:** gli ID devono essere `int32`; le distanze devono essere coerenti con la versione (`float` per 32, `double` per 64/64omp) e salvate in row-major.
3. **Range degli ID:** ogni ID deve essere compreso in $[0, N - 1]$ dove N è la cardinalità del dataset.
4. **Allineamento e integrità:** i file devono essere completamente leggibili senza offset errati; eventuali mismatch nella dimensione del payload indicano scrittura errata (ad esempio ID a 8 byte).

5.4 Criteri di uguaglianza e tolleranze

Il confronto sugli ID è tipicamente un confronto discreto (uguaglianza esatta), mentre le distanze possono differire leggermente a causa di:

- differenze di precisione numerica (`float` vs `double`);
- ordine delle operazioni (riduzioni SIMD e, in 64omp, differenze minime dovute al scheduling).

Per questo motivo, quando si confrontano le distanze è ragionevole considerare una tolleranza numerica. In pratica, le differenze più rilevanti che possono comparire sono:

- in 32: errori relativi più alti (singola precisione);
- in 64/64omp: maggiore stabilità numerica.

L'obiettivo del `compare results` è quindi assicurare che l'output sia compatibile, coerente e stabile, e che eventuali differenze rientrino nelle attese dovute al tipo numerico e alle ottimizzazioni.

5.5 Workflow di confronto e significato di PASS

La verifica viene eseguita tramite lo script di test del progetto, che: (i) carica dataset e query in formato `.ds2`, (ii) esegue `fit` e `predict`, (iii) salva i risultati nei file `out_idnn.ds2` e `out_distnn.ds2`, (iv) confronta struttura e contenuti degli output con i riferimenti attesi.

Un test viene considerato **PASS** se:

- i file prodotti sono leggibili e rispettano dimensioni e formato `.ds2` (header corretto e payload coerente);
- gli ID hanno tipo `int32` e sono nel range $[0, N - 1]$;
- le distanze sono numericamente coerenti con i risultati attesi, tenendo conto delle differenze di precisione (`float` vs `double`) e dell'ordine delle operazioni.

In particolare, per garantire piena compatibilità con il framework del docente, nelle versioni 64 e 64omp gli ID vengono forzati a 4 byte tramite `save_int_data`.

Capitolo 6

Test, validazione ed edge cases

Oltre alla correttezza sui casi standard, un progetto destinato a essere valutato tramite esecuzione automatica deve essere robusto rispetto a input limite e configurazioni non ideali. In questo capitolo descriviamo i test eseguiti per verificare: (i) compatibilità dei formati di input/output richiesti, (ii) comportamento con configurazioni non ideali, (iii) stabilità numerica e assenza di crash, (iv) coerenza tra versioni 32, 64, 64omp.

6.1 Obiettivo dei test e criteri di robustezza

Oltre alla correttezza funzionale sul caso standard, il progetto deve risultare robusto in condizioni operative realistiche, soprattutto perché la valutazione avviene tramite esecuzione automatica. In questa fase non si tratta di introdurre nuovi requisiti rispetto alla specifica, ma di verificare che l'implementazione:

- produca sempre output compatibili con il formato `.ds2` richiesto;
- non presenti crash o accessi fuori range in presenza di dimensioni non “comode” per la SIMD (gestione del *tail*);
- gestisca correttamente l'allineamento dei buffer quando si usano load/store allineati;
- sia thread-safe nella versione 64omp (assenza di race condition) e mantenga risultati stabili.

Assumiamo parametri validi come da specifica (ad esempio $k > 0$, $h > 0$, $0 \leq x \leq D$) e concentriamo i test sugli edge cases che emergono in modo naturale nell'implementazione ad alte prestazioni (SSE/AVX/OpenMP) e nel formato I/O.

6.2 Assunzioni sui parametri e casi limite realistici

Nel contesto di valutazione automatica assumiamo parametri validi come da specifica del progetto (ad esempio $k > 0$, $h > 0$, $0 \leq x \leq D$). I test su edge cases sono quindi

concentrati sui casi limite che emergono *realmente* in implementazioni ad alte prestazioni (SIMD/OpenMP) e nel formato dei file.

6.2.1 Dimensioni non allineate alla SIMD (gestione del tail)

Le routine SSE/AVX elaborano i dati a blocchi (4 `float` per SSE, 4 `double` per AVX). Quando D non è multiplo della larghezza SIMD, la parte rimanente (*tail*) viene gestita scalarmente. Sono stati testati valori di D non multipli della SIMD per verificare che:

- non si verifichino accessi fuori range;
- i risultati siano coerenti con la versione C.

6.2.2 Allineamento dei buffer e robustezza del wrapper

L'uso di load/store allineati richiede che i buffer siano allocati con l'allineamento previsto (16 byte in 32, 32 byte in 64/64omp). Sono stati verificati i controlli lato wrapper, in modo che input non allineati vengano intercettati con un errore gestito, evitando crash o comportamento indefinito.

6.2.3 Robustezza multi-thread (versione 64omp)

Per la versione parallela, sono stati eseguiti test variando il numero di thread (`OMP_NUM_THREADS`) e ripetendo più volte l'esecuzione sugli stessi input, con l'obiettivo di verificare:

- assenza di race condition e crash;
- stabilità dei risultati (eventuali differenze minime sono attribuibili a pari-merito o a dettagli numerici).

6.2.4 Dataset di piccole dimensioni per validazione funzionale

Sono stati utilizzati dataset e query di dimensioni ridotte per facilitare la validazione funzionale e l'ispezione manuale degli output, verificando in modo diretto:

- range degli ID e coerenza tra ID e distanze;
- correttezza del formato `.ds2` in lettura e scrittura.

6.3 Esiti dei test su dimensioni non allineate alla SIMD

Le esecuzioni con D non multiplo della larghezza SIMD (SSE/AVX) hanno prodotto output corretti e file `.ds2` validi in tutte le versioni. In particolare, non sono stati osservati crash né accessi fuori range, confermando la corretta gestione della parte residua (*tail*) dopo il loop vettorizzato. I risultati ottenuti sono coerenti con la versione C, al netto delle differenze attese dovute al tipo numerico (`float` vs `double`).

6.4 Esiti dei test di robustezza multi-thread (64omp)

Ripetendo l'esecuzione della versione `64omp` variando il numero di thread (`OMP_NUM_THREADS`), il programma ha mantenuto stabilità e correttezza degli output. L'assenza di race condition è supportata dall'uso di strutture allocate per-thread (ad esempio buffer locali per `knn` e `qpivot`), che riducono la necessità di sincronizzazione. Eventuali variazioni minime possono essere attribuite a dettagli numerici o a casi di pari-merito nella selezione dei vicini, e non compromettono la validazione in `PASS`.

6.5 Coerenza tra versioni

Infine, vengono confrontate le tre versioni (`32`, `64`, `64omp`) su piccoli dataset controllati per verificare:

- correttezza strutturale degli output (shape e formato);
- differenze attese dovute a precisione (`float` vs `double`);
- equivalenza logica della pipeline (stesso comportamento generale, pruning coerente).

Questa batteria di test consente di intercettare errori tipici di progetti ad alte prestazioni (out-of-bounds, errori di formato, non thread-safety, gestione errata della coda SIMD) prima dell'esecuzione automatica del docente.

Capitolo 7

Benchmark e benchmark_dimensions

La valutazione del progetto non dipende solo dalla correttezza, ma soprattutto dalle prestazioni. In questo capitolo descriviamo la metodologia di benchmark e l'analisi delle misure ottenute, con particolare attenzione a: (i) confronto tra C e assembly, (ii) confronto tra versioni 32/64/64omp, (iii) scalabilità al variare delle dimensioni del problema (benchmark_dimensions).

7.1 Metodologia sperimentale

Per rendere significativi i tempi misurati, i benchmark seguono queste regole:

- **Warm-up:** una prima esecuzione elimina effetti di cold cache e JIT/overhead iniziali.
- **Ripetizioni:** ogni misura viene ripetuta più volte e si riportano media e/o minimo (a seconda della convenzione adottata dal docente).
- **Separazione fit/predict:** si distinguono i tempi di indicizzazione (`fit`) e quelli di interrogazione (`predict`), poiché incidono in modo diverso nei carichi reali.
- **Parametri fissati:** per confronti corretti, `h`, `k`, `x` vengono mantenuti costanti, variando una sola dimensione per volta nei benchmark_dimensions.

7.2 Benchmark “compare” sulle istanze del docente

Il benchmark principale replica lo scenario di valutazione: dataset e query forniti, parametri coerenti con lo script di test, generazione degli output e raccolta dei tempi. In questa fase il confronto è:

- **C vs ASM** (stessa versione): evidenzia il guadagno SIMD sulle routine di distanza/lower bound;

- **32 vs 64**: evidenzia differenze tra singola e doppia precisione e tra SSE e AVX;
- **64 vs 64omp**: evidenzia il beneficio della parallelizzazione OpenMP.

7.2.1 Cosa misuriamo

Le metriche principali sono:

- **Tempo di fit**: include quantizzazione dataset e costruzione indice pivot.
- **Tempo di predict**: include quantizzazione query, pruning, selezione candidati e raffinamento euclideo.
- **Tempo totale**: somma di fit e predict (utile per scenari in cui si rifà fit spesso).

7.3 Benchmark_dimensioni: scalabilità con N e D

Il benchmark_dimensioni mira a studiare come crescono tempi e speedup al variare delle dimensioni:

- **scalabilità con N** (numero di punti): impatta soprattutto lo scanning in predict e la dimensione dell'indice;
- **scalabilità con D** (dimensione dei vettori): impatta direttamente il costo delle distanze e la quantizzazione;
- **scalabilità con n_q** (numero di query): utile per valutare l'efficacia dell'OpenMP (parallelizzazione per query).

7.3.1 Aspettative teoriche (coerenti con l'implementazione)

- **Aumentando D** : cresce il costo delle distanze; ci si aspetta che SSE/AVX aumentino il beneficio relativo perché la parte vettorizzabile domina.
- **Aumentando N** : cresce il numero di candidati da valutare; l'efficacia del pruning (pivot + lower bound) influisce molto.
- **Aumentando n_q** : nella versione 64omp cresce il parallelismo disponibile; ci si aspetta uno speedup più evidente.

7.4 Interpretazione dei risultati

I risultati vanno letti tenendo presenti i colli di bottiglia:

- in **fit** dominano quantizzazione dataset e costruzione dell'indice (molti accessi in memoria e molte distanze approssimate);
- in **predict** domina lo scanning del dataset con pruning + le distanze euclidee sui candidati finali;
- in **64omp** il guadagno dipende anche da scheduling e bilanciamento: la scelta `schedule(dynamic,10)` riduce squilibri tra query più/meno costose.

7.4.1 Come riportiamo i risultati in relazione

Per rendere la valutazione chiara e confrontabile, consigliamo di inserire:

- una tabella con tempi **fit**, **predict** e **totale** per ciascuna versione (C/ASM);
- una tabella di **speedup** (es. SSE vs C32, AVX vs C64, 64omp vs 64);
- una tabella o grafico per benchmark_dimensioni mostrando l'andamento con N e D .

7.5 Discussione dei risultati

Dai benchmark emerge un miglioramento netto passando dalle versioni C alle versioni con assembly SIMD: nella versione 32 l'uso di SSE riduce il tempo di **predict** da 5.408s a 1.776s (circa $3.0\times$), mentre nella 64 l'uso di AVX riduce **predict** da 6.589s a 1.586s (circa $4.2\times$). Questo comportamento è coerente con il fatto che i colli di bottiglia principali risiedono nei loop di distanza (approssimata/euclidea) e nel calcolo del lower bound, che beneficiano direttamente della vettorizzazione.

Per quanto riguarda la versione parallela, **64omp** introduce un ulteriore beneficio distribuendo tra thread l'elaborazione delle query (`schedule(dynamic,10)`), riducendo **predict** a 3.664s già in C+OpenMP (circa $1.8\times$ rispetto alla baseline 64 C). Combinando OpenMP con AVX (AVX+OpenMP) si ottiene il miglior risultato complessivo sul **predict**, pari a 1.331s (circa $4.9\times$ rispetto a 64 C).

È importante notare che lo *speedup* riportato nella tabella “Speedup vs C” è calcolato sul tempo di **predict** (PRD), in accordo con lo script di benchmark. Il tempo di **fit** può invece risentire maggiormente dell'overhead di parallelizzazione e della struttura dell'indice; ciò spiega perché, in alcuni casi, **fit** non scala in modo monotono pur migliorando significativamente la fase **predict**, che è tipicamente la componente dominante nelle applicazioni di querying.

#	Versione	Tipo	Esito	FIT (s)	PRD (s)	Totale (s)	Speedup PRD
1	32-bit	C Baseline	PASS	0.104	5.408	5.512	1.0×
2	32-bit	SSE Assembly	PASS	0.078	1.776	1.854	3.0×
3	64-bit	C Baseline	PASS	0.116	6.589	6.705	1.0×
4	64-bit	AVX Assembly	PASS	0.064	1.586	1.650	4.2×
5	64omp	C + OpenMP	PASS	0.069	3.664	3.733	1.8×
6	64omp	AVX + OpenMP	PASS	0.134	1.331	1.465	4.9×

In conclusione, i benchmark dimostrano l'efficacia delle ottimizzazioni: SIMD riduce il costo delle routine di distanza e del lower bound, mentre OpenMP consente un'ulteriore accelerazione distribuendo tra thread l'elaborazione delle query e le fasi indipendenti del fit.